

Ottimizzazione con algoritmi *surrogate* in Python

Davide Angelini
davide.angelini@polito.it



**Politecnico
di Torino**

Dipartimento
di Ingegneria Meccanica
e Aerospaziale



Outline

- **Modulo 1**
 - Paradigmi di programmazione: OOP, SoC, DRY
 - Algoritmo surrogate con RBFOpt
 - Applicazioni pratiche dell'algoritmo
- Modulo 2 (prossima lezione)
 - Connessione tra Python e software esterni (es.: Patran e Nastran)
 - Ottimizzazione di un modello trave

Outline

- **Modulo 1**

Paradigmi di programmazione: OOP, SoC, DRY

- Algoritmo surrogate con RBFOpt
- Applicazioni pratiche dell'algoritmo

- **Modulo 2 (prossima lezione)**

- Connessione tra Python e software esterni (es.: Patran e Nastran)
- Ottimizzazione di un modello trave

Perché il codice mal strutturato è costoso

In ambienti di ricerca o innovativi (ricerca in accademia, ricerca industriale, startup, ecc...) accade che le specifiche di un progetto vengano aggiornate frequentemente, in base a nuovi dati o risultati che arrivano. Ciò porta i codici informatici a essere velocemente obsoleti e a doverli riscrivere da capo, oltre al possibile accumulo di bug.

Scenario	Codice mal strutturato	Codice ben strutturato
Devo cambiare una funzione	Riscrivo tutto il codice	Cambio un solo file
C'è un bug	Non capisco bene dove trovarlo	So già in quale modulo trovarlo (lo dice il codice stesso)
Un collega riusa il mio codice	Fa copia e incolla delle parti, lasciando parti aggiornate e deprecate	Fa direttamente <i>import</i> del mio codice
Riprendo il progetto dopo un periodo di stop (es.: ferie)	Devo rileggere tutto da capo per ripartire	La struttura è chiara e parla da sé

Perché il codice mal strutturato è costoso

Esistono tre filosofie di programmazione che rendono il codice più modulabile, adattabile a diversi contesti e soprattutto sintetico (dal greco *synthesis*, unire gli argomenti):

- **Programmazione a oggetti** (Object Oriented Programming)
- **Separazione di ruoli** (Separation of Concerns)
- **DRY** (Don't Repeat Yourself)

Object Oriented Programming organizza il codice attorno a **classi** (il progetto) e **oggetti** (le istanze).

Separation of Concerns: ogni modulo, classe o file ha **una sola responsabilità** ben definita.

Don't Repeat Yourself: ogni informazione deve avere **un'unica rappresentazione** nel sistema.

Programmazione a Oggetti

La **Programmazione a Oggetti** organizza il codice attorno a **classi** (il progetto) e **oggetti** (le istanze). Ogni oggetto ha:

- **Attributi** → lo stato interno
- **Metodi** → le azioni che può compiere

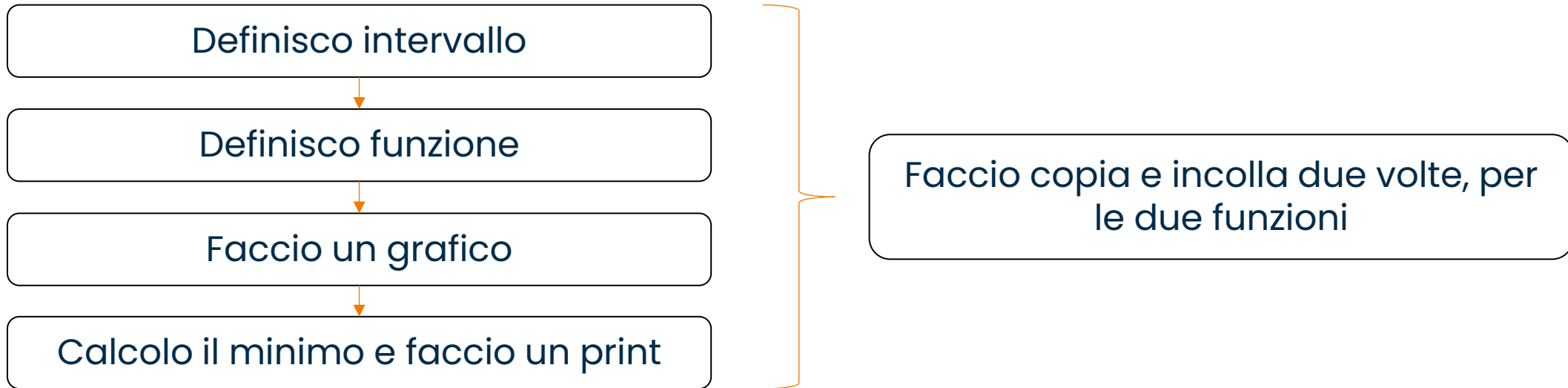


Vediamolo con un esempio.

Programmazione a Oggetti

Voglio trovare il minimo delle funzioni $y=\sin(x)$ e $y=x^2$ all'interno dell'intervallo $[0, 2\pi]$ e voglio una rappresentazione grafica.

Approccio *sbagliato*:



Se poi devo fare ulteriori modifiche, devo modificare il codice in più parti e per ogni funzione... mi perdo e perdo tempo. Vediamo un esempio invece secondo OOP (vedi file esempio_oop.py)

Programmazione a Oggetti

Secondo OOP, definisco una classe astratta con: una proprietà molto generica di *evaluate*; una proprietà di *plot* che richiama l'*evaluate* e costruisce il grafico.

```
# — Classe base: definisce il "contratto"
class ObjectiveFunction:
    """
    Qualsiasi funzione obiettivo deve saper fare queste due cose:
    * valutazione (evaluate)
    * un grafico (plot).
    """

    def evaluate(self, x: np.ndarray) -> np.ndarray:
        raise NotImplementedError

    def plot(self, x: np.ndarray) -> None:
        y = self.evaluate(x)
        plt.plot(x, y, label=self.__class__.__name__)
        plt.title(self.__class__.__name__)
        plt.grid()
        plt.xlabel('Asse X')
        plt.show()
```


Programmazione a Oggetti

A questo punto, ogni funzione successiva richiama la mia `ObjectiveFunction`, ereditandone le proprietà, e sovrascrive la proprietà di `evaluate` con un `evaluate` proprio (che sarebbe $\sin(x)$ o x^2).

Quando richiamerò la funzione `SinFunction`, in «automatico» avrà già tutte le proprietà di plot definite!

Se voglio modificare il plot e cambiare i colori, farò una sola modifica nella classe `ObjectiveFunction` e tutto il codice erediterà la modifica.

```
class SinFunction(ObjectiveFunction): #<-- Qui dico che voglio rispettare il "contratto" di sopra, ObjectiveFunction
    ... def evaluate(self, x: np.ndarray) -> np.ndarray:
    ... | ... return np.sin(x)
    ...
class ParabolaFunction(ObjectiveFunction):
    ... def evaluate(self, x: np.ndarray) -> np.ndarray:
    ... | ... return x ** 2
```

Programmazione a Oggetti

A questo punto, ogni funzione successiva richiama la mia `ObjectiveFunction`, ereditandone le proprietà, e sovrascrive la proprietà di `evaluate` con un `evaluate` proprio (che sarebbe $\sin(x)$ o x^2).

Quando richiamerò la funzione `SinFunction`, in «automatico» avrà già tutte le proprietà di plot definite!

Se voglio modificare il plot e cambiare i colori, farò una sola modifica nella classe `ObjectiveFunction` e tutto il codice erediterà la modifica.

```
class SinFunction(ObjectiveFunction): #<-- Qui dico che voglio rispettare il "contratto" di sopra, ObjectiveFunction
    ... def evaluate(self, x: np.ndarray) -> np.ndarray:
    ... | ... return np.sin(x)
    ...
class ParabolaFunction(ObjectiveFunction):
    ... def evaluate(self, x: np.ndarray) -> np.ndarray:
    ... | ... return x ** 2
```

Separation of Concerns

Separation of Concerns: ogni modulo, classe o file ha **una sola responsabilità** ben definita.

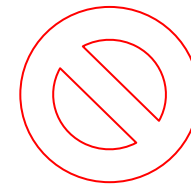
Regola pratica: se descrivi cosa fa un file e usi la parola "**e**", probabilmente sta facendo troppo.

Funzione 1 deve fare ABCD

Funzione 2 deve fare XYZ



Funzione 1 deve fare ABCD **e** XYZ



Separation of Concerns

Nell'esempio fornito, costruisco un'altra funzione che trova il minimo usando argmin di numpy. Come vedete non la funzione non sa cosa riceve, vede solo che riceverà una classe ObjectiveFunction e che dovrà applicare argmin su evaluate. Poi non sa se quell'evaluate richiederà un $\sin(x)$, un x^2 o altro ma non importa: **la sua responsabilità è applicare unicamente argmin.**

```
# — Funzione che ottimizza: non sa cosa riceve, non gliene importa
def find_minimum(func: ObjectiveFunction, x: np.ndarray) -> float:
    ... return x[np.argmin(func.evaluate(x))]

    ... # Provatelo a sostituire questo return con il seguente: il codice in automatico cambierà metodo
    ... # di minimizzazione per tutte le funzioni scritte, basta modificare una sola riga
    ... # result = minimize_scalar(np.sin, bounds=(0, 2*np.pi), method='bounded')
    ... # return result.x
```

Don't Repeat Yourself

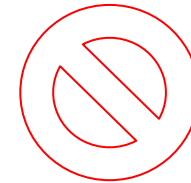
Don't Repeat Yourself: ogni informazione deve avere **un'unica rappresentazione** nel sistema.

Se la stessa logica appare in due posti, quando devi correggerla la correggi in uno solo e introduci un bug nell'altro.

Scrivere le funzioni una sola volta e
importarle nei vari file



Fare copia e incolla



Outline

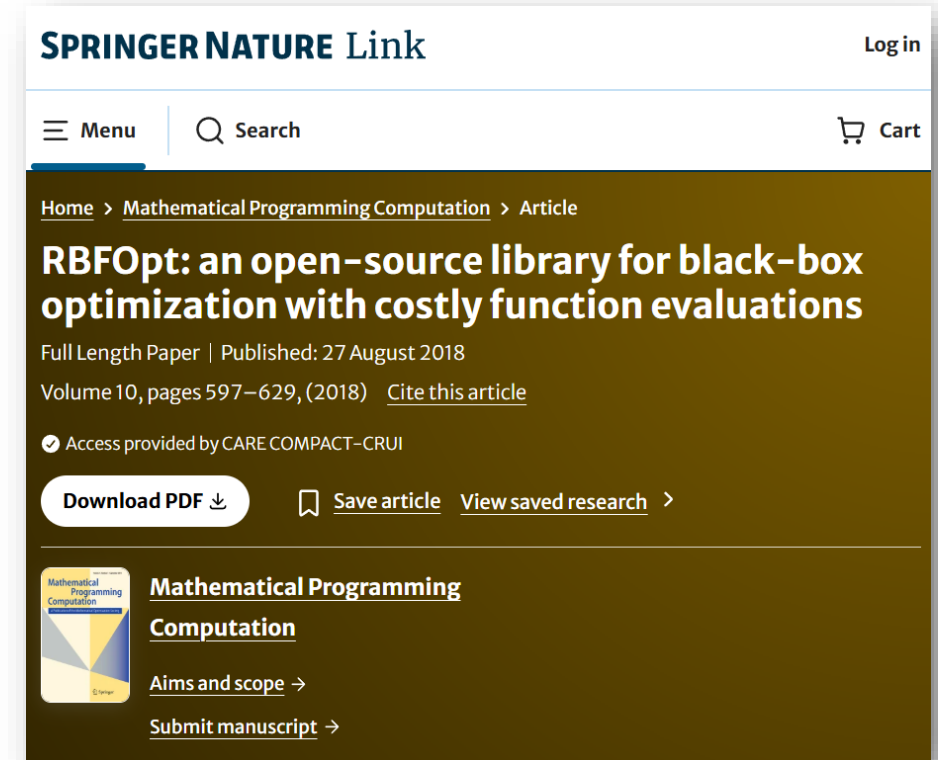
- **Modulo 1**
 - Paradigmi di programmazione: OOP, SoC, DRY
 - **Algoritmo surrogate con RBFOpt**
 - Applicazioni pratiche dell'algoritmo
- Modulo 2 (prossima lezione)
 - Connessione tra Python e software esterni (es.: Patran e Nastran)
 - Ottimizzazione di un modello trave

Algoritmo surrogate con *RBFOpt*

RBFOpt è una libreria per **Python** completamente gratuita e open source, creata da Costa & Nannicini [1].

Consente di risolvere problemi di ottimizzazione con un numero molto ristretto di valutazioni della funzione obiettivo.

La libreria costruisce un **surrogato della funzione reale**, con il vincolo che nei punti valutati il valore della funzione reale deve essere uguale al valore della funzione surrogata. Punta dunque a cercare il **minimo del surrogato**, che corrisponderà con il minimo della funzione reale. Dal paper originale la libreria si è evoluta, è continuamente tenuta attiva e consente un livello di personalizzazione che più difficilmente si raggiunge con altri linguaggi «chiusi» (es.: Matlab).



Costa, A., Nannicini, G. *RBFOpt: an open-source library for black-box optimization with costly function evaluations*. *Math. Prog. Comp.* 10, 597–629 (2018). <https://doi.org/10.1007/s12532-018-0144-7>

Algoritmo surrogate con *RBFOpt*

Idea di fondo: trovare il minimo di $f(X)$, con $X=[x_1, x_2, x_3, \dots, x_N]$, valutando questa funzione il minor numero di volte possibili.

Strategia in sintesi:

- Valuto $f(x)$ in $N+1$ punti iniziali
- Costruisco un surrogato di $f(x)$
- Decido quali sono i prossimi migliori punti x_{new} su cui valutare $f(x_{\text{new}})$ per trovare il minimo

$$s_k(x) := \sum_{i=1}^k \lambda_i \phi(\|x - x_i\|) + p(x),$$

Base radiale

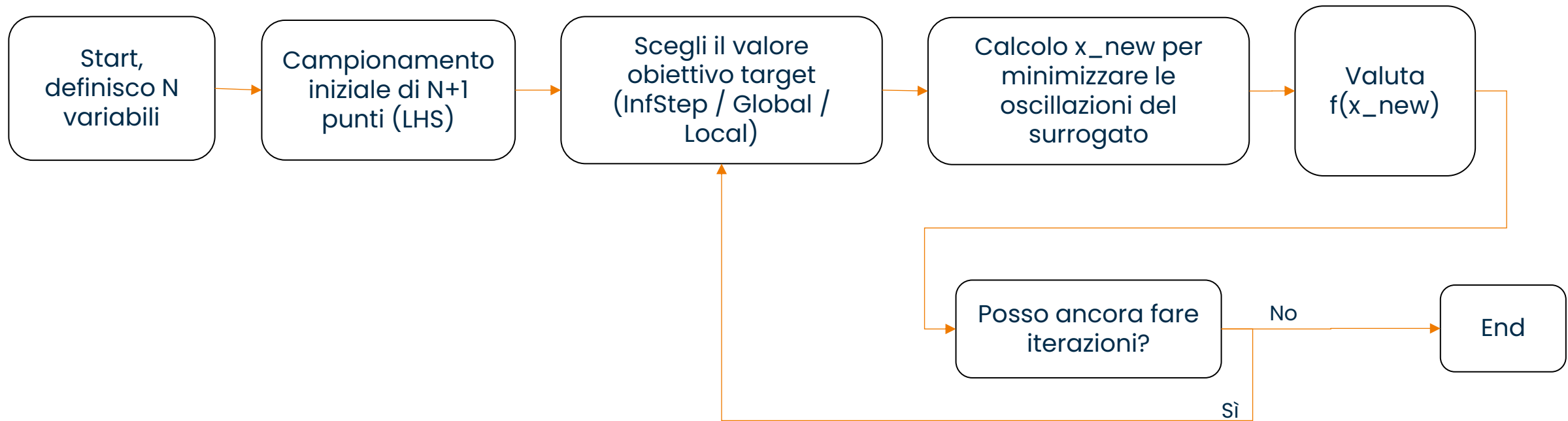
Punti valutati

Termine
polinomiale

$\phi(r)$		$d_{\min}$
r	(linear)	0
r^3	(cubic)	1
$\sqrt{r^2 + \gamma^2}$	(multiquadric)	0
$r^2 \log r$	(thin plate spline)	1

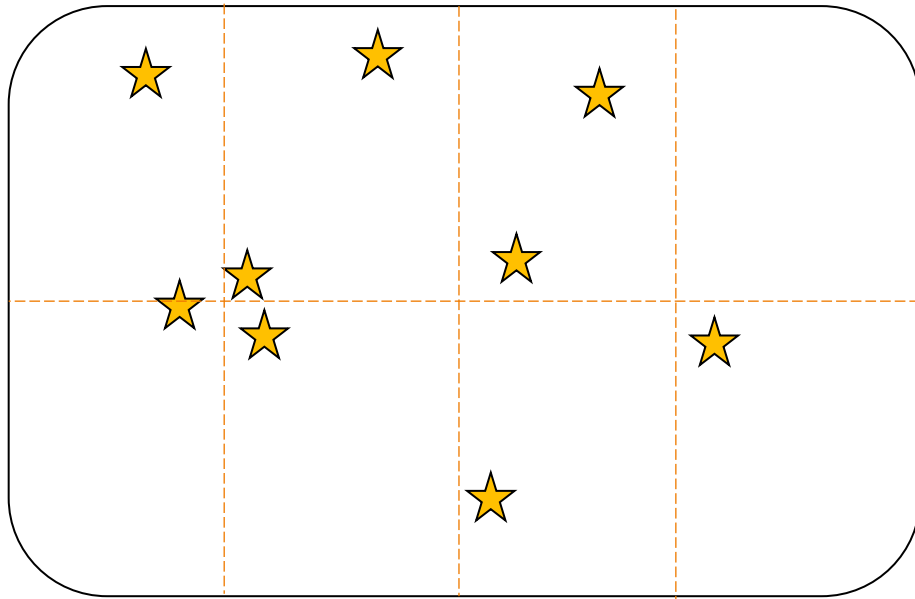
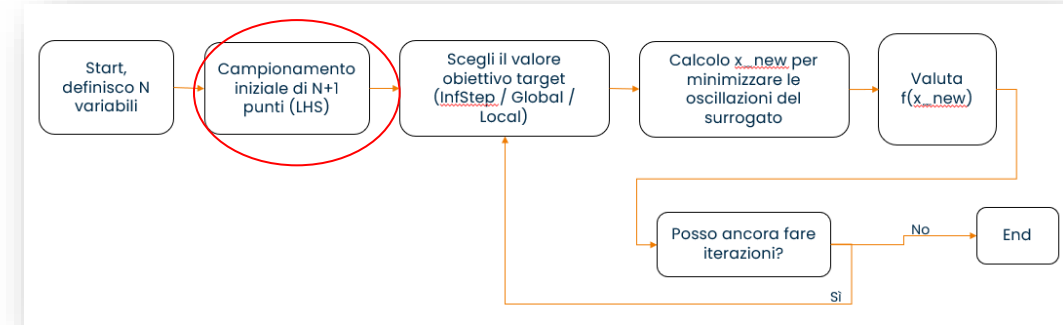
Algoritmo surrogate con *RBFOpt*

Un grande vantaggio è che **gli input iniziali possono essere sia numeri reali, sia numeri discreti**. Lo schema generale di funzionamento è il seguente, vedremo nelle slide successive il significato dei termini.

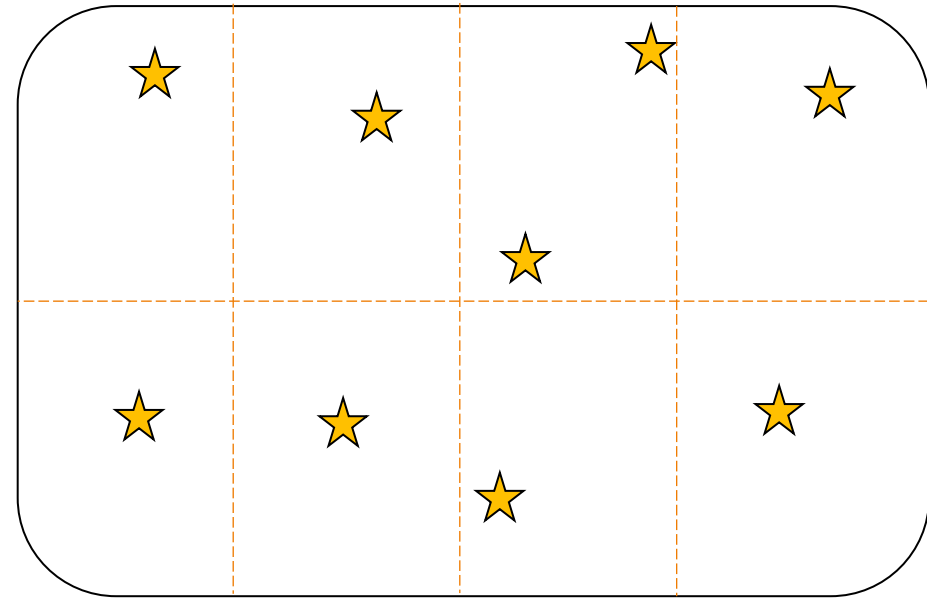


Algoritmo surrogate con *RBFOpt*

Come scelgo gli $N+1$ punti iniziali? Uso un algoritmo interno chiamato **Latin Hypercube Sample (LHS)**. Dividi il dominio in $N+1$ intervalli uguali per ogni dimensione e campiona un punto per intervallo, garantendo che nessuna parte del dominio sia ignorata. È superiore al campionamento casuale puro perché copre lo spazio in modo più uniforme con pochi punti.



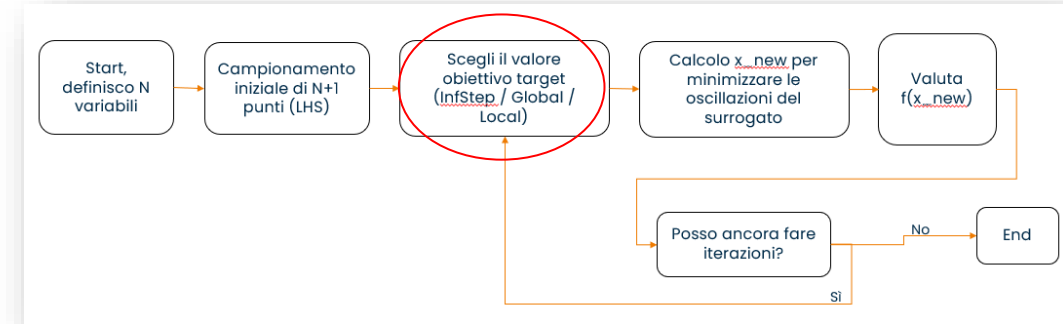
Punti casuali



Punti con LHS

Algoritmo surrogate con *RBFOpt*

Dato che la funzione $f(x)$ è *costosa* da valutare, come scelgo i punti da valutare? Ho due strategie.



Exploration

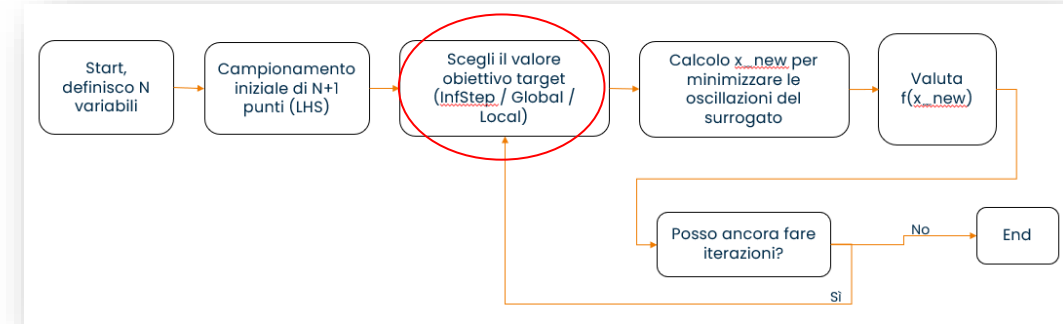
Valuto punti *lontani* da quelli già noti.
Rischio: spreco valutazioni in zone inutili.
Beneficio: non mi perdo il minimo globale.

Exploitation

Valuto punti *vicino* al minimo corrente.
Rischio: mi blocco in un minimo locale.
Beneficio: convergo velocemente se il minimo locale è quello globale.

Algoritmo surrogate con *RBFOpt*

Si individuano tre tipologie di iterazioni: *InfStep*, *globalStep*, *localStep*. Altre (come i *RefinementStep*) sono varianti di queste



infStep,

Esplorazione pura, trova punti lontani da tutti i valutati

Lo faccio a inizio ciclo

globalStep,

Bilancia exploration ed exploitation

Ciclo principale

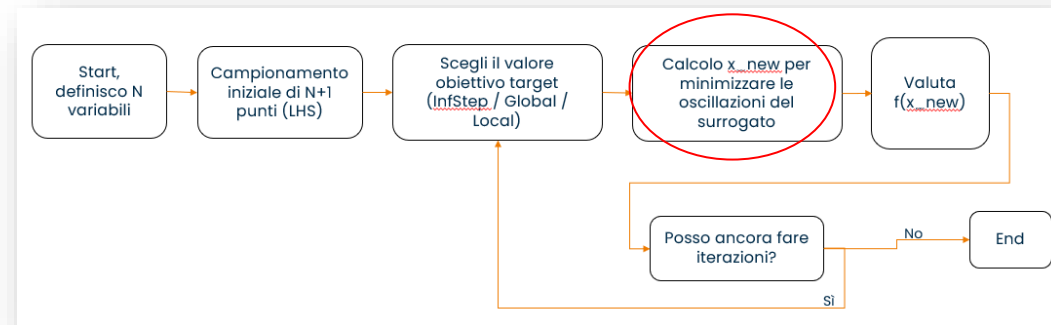
localStep,

Puro exploitation: cerca il minimo nell'intorno del punto migliore sinora trovato

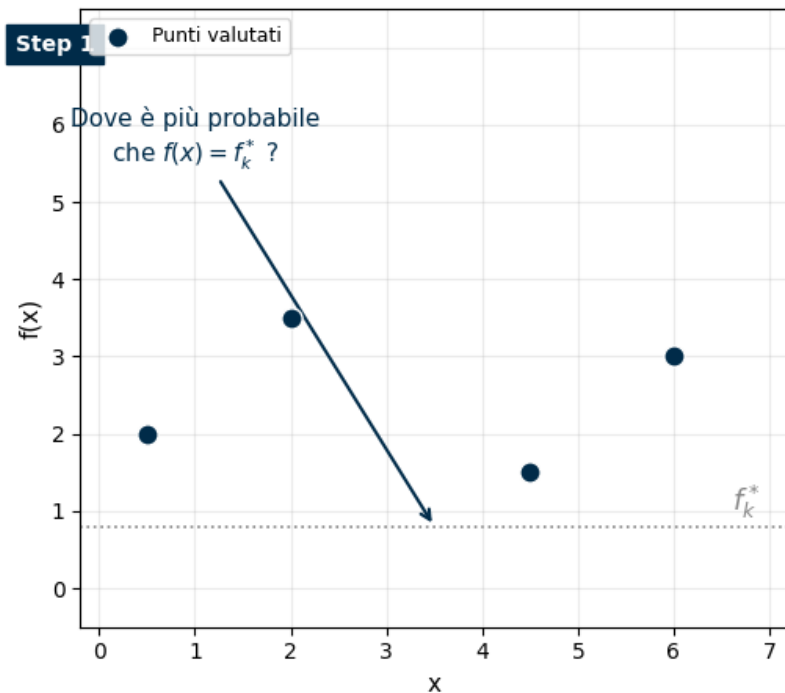
Fine ciclo

Algoritmo surrogate con *RBFOpt*

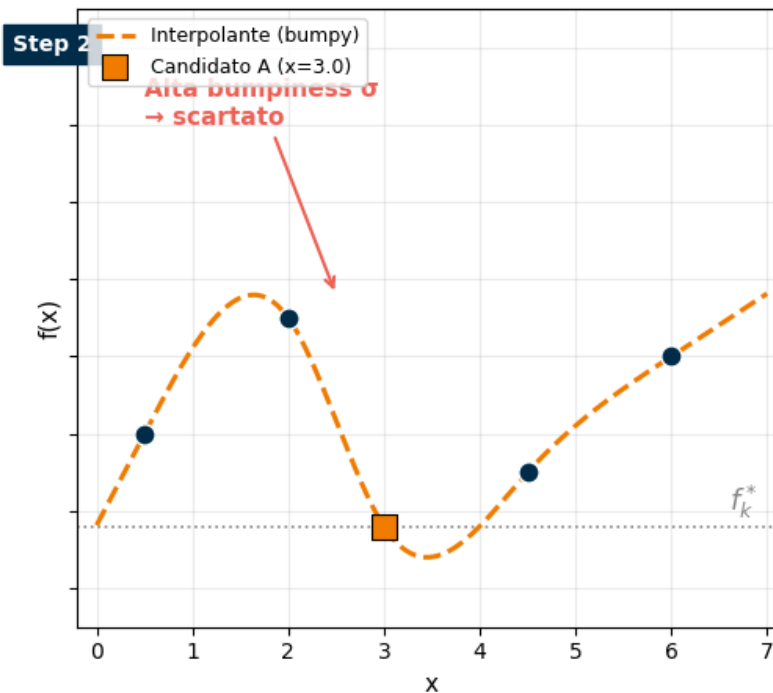
Come valuto il prossimo candidato? Minimizzando le oscillazioni del surrogato e i cambi di concavità.



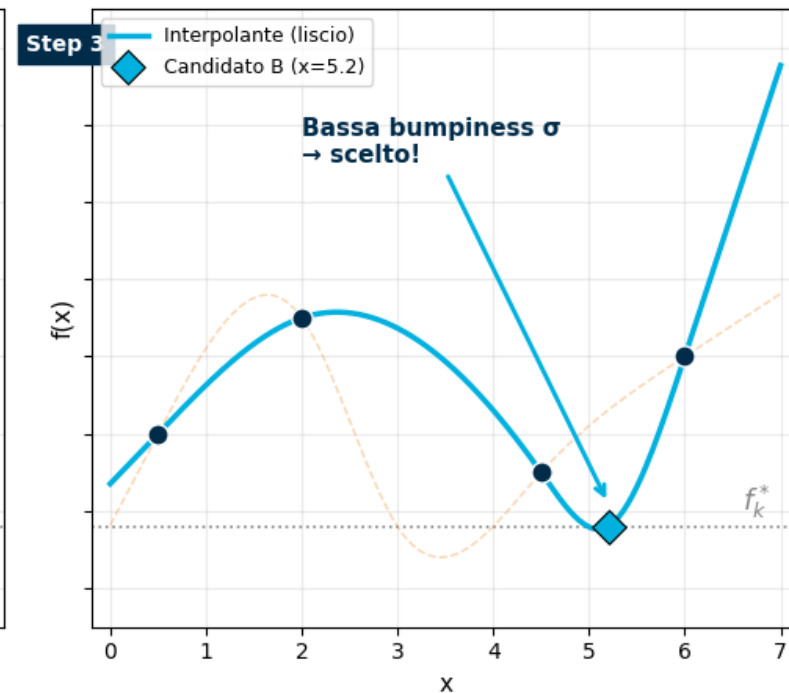
Punti valutati e target f_k^*



Candidato A: interpolante bumpy



Candidato B: interpolante liscio ✓



Algoritmo surrogate con *RBFOpt*

Posso ottimizzare problemi di ogni tipo. In questa ricerca abbiamo creato un generatore di mesh ad alta fedeltà che copre dalla scala millimetrica a nanometrica in pochi passi.

Posso scegliere a mio piacimento i valori di penalty da dare in caso di elementi distorti.

Il codice era stato scritto secondo i tre paradigmi di prima ed è adattabile al 3D.

PAPER • OPEN ACCESS

High-Fidelity Meshing and Stochastic Simulation for Enhanced Surface Stress Concentration Prediction

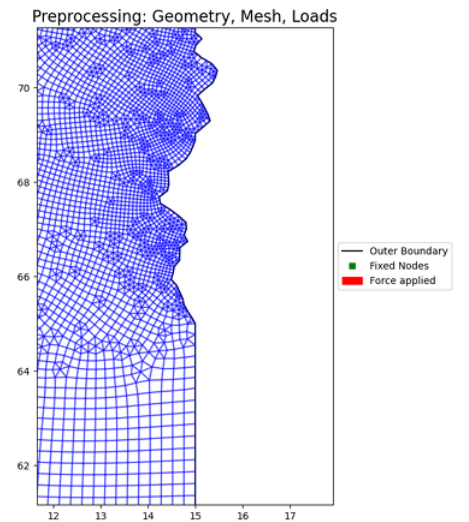
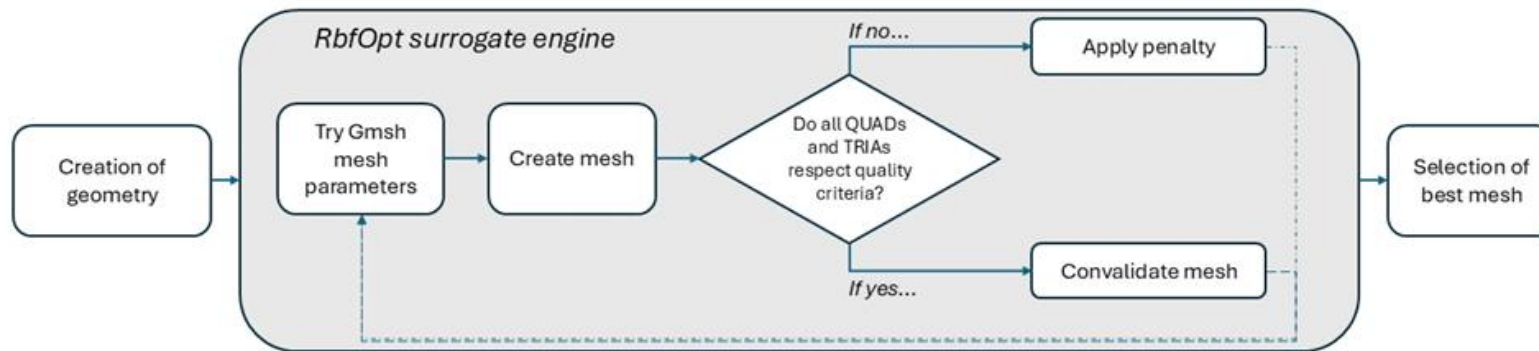
Davide Angelini, Enrico Cestino, Sergio Bianco and Fabio Mallamo

Published under licence by IOP Publishing Ltd

[IOP Conference Series: Materials Science and Engineering, Volume 1338, 45th Risø International Symposium on Materials Science: Advancement in composites through characterisation, modelling and digitalisation 01/09/2025 - 04/09/2025 Copenhagen, Denmark](#)

Citation Davide Angelini et al 2025 IOP Conf. Ser.: Mater. Sci. Eng. 1338 012011

DOI 10.1088/1757-899X/1338/1/012011



Applicazioni pratiche dell'algoritmo

- Elementi essenziali sono:
 - Valori minimi e massimi che le variabili da ottimizzare devono assumere, definiti in *var_lower* e *var_upper*
 - Tipologia di variabile («R» = reale, «I» = integer)
 - Valutazioni massime da fare, ovvero *max_evaluations*
Consiglio: almeno $30 * (N+1)$ valutazioni

Altre personalizzazioni da valutare:

- Tipologia di surrogato (lineare, cubico, ecc...)
- Filtro logaritmico da applicare ai valori
- Numero obbligatorio di global research

```
# Definisce il problema black-box per RBFopt
problema = rbfopt.RbfoptUserBlackBox(
    ... dimension=1,
    ... var_lower=np.array([X_LOWER]),
    ... var_upper=np.array([X_UPPER]),
    ... var_type=np.array(["R"]),
    ... obj_funct=self._valuta_funzione,
)
```

```
# Impostazioni dell'algoritmo RBFopt
impostazioni = rbfopt.RbfoptSettings(
    ... max_evaluations=MAX_VALUTAZIONI,
    ... rbf=KERNEL_RBF,
    ... minlp_solver_path=percorso_bonmin,
    ... nlp_solver_path=percorso_ipopt,
)
```

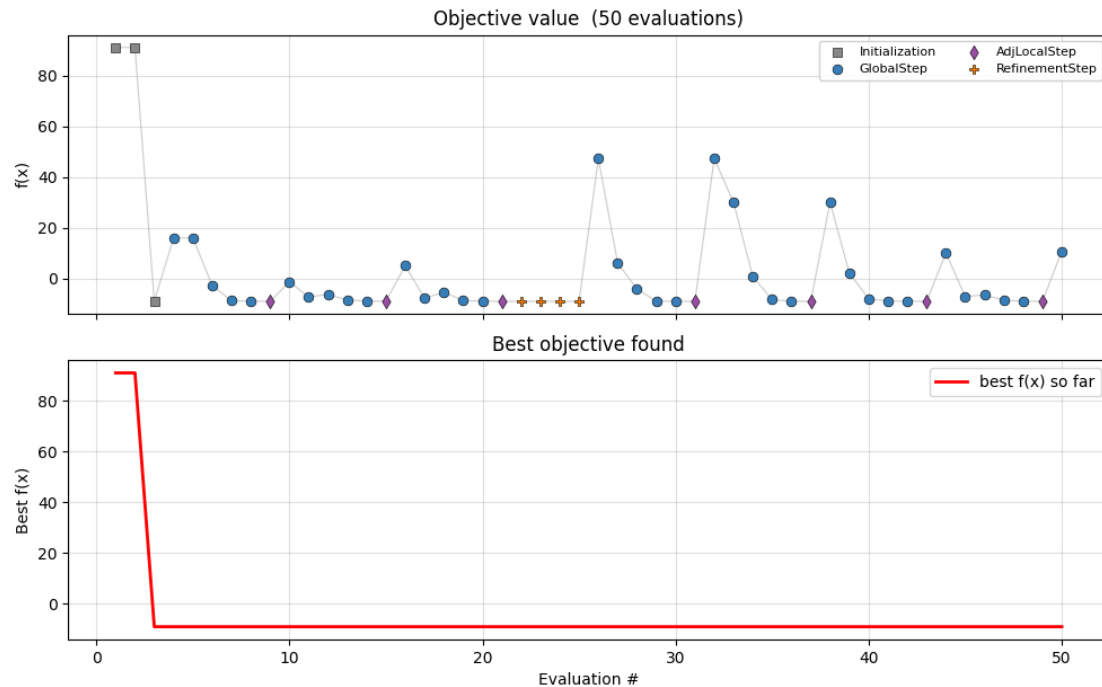
```
# Crea l'algoritmo e intercetta il suo output per catturare i tipi di passo
algoritmo = rbfopt.RbfoptAlgorithm(impostazioni, problema)
intercettore = LogInterceptor(sys.stdout, self._gestore)
algoritmo.set_output_stream(intercettore)

# Avvia l'ottimizzazione
val_ottimo, x_ottimo, n_iterazioni, n_valutazioni, _ = algoritmo.optimize()
```

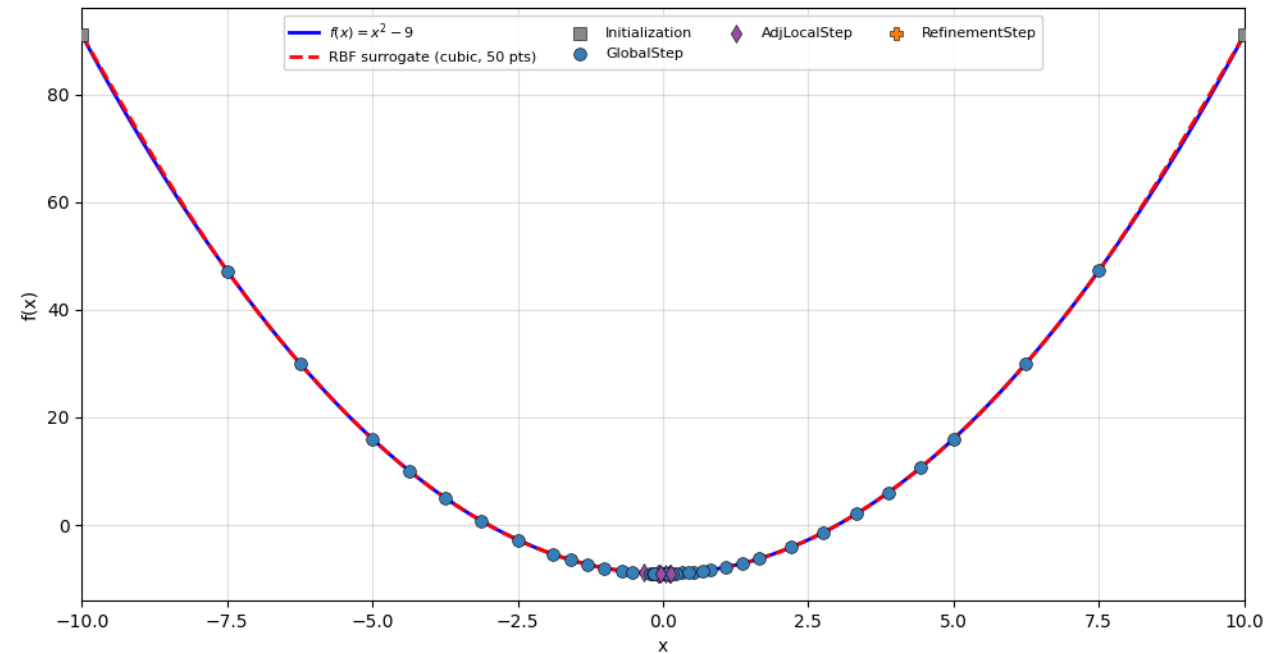
Applicazioni pratiche dell'algoritmo

Esempio 1: minimizzare $y = x^2 - 9$
Convergenza molto veloce
perché funzione polinomiale.

RBFOpt - live convergence monitor



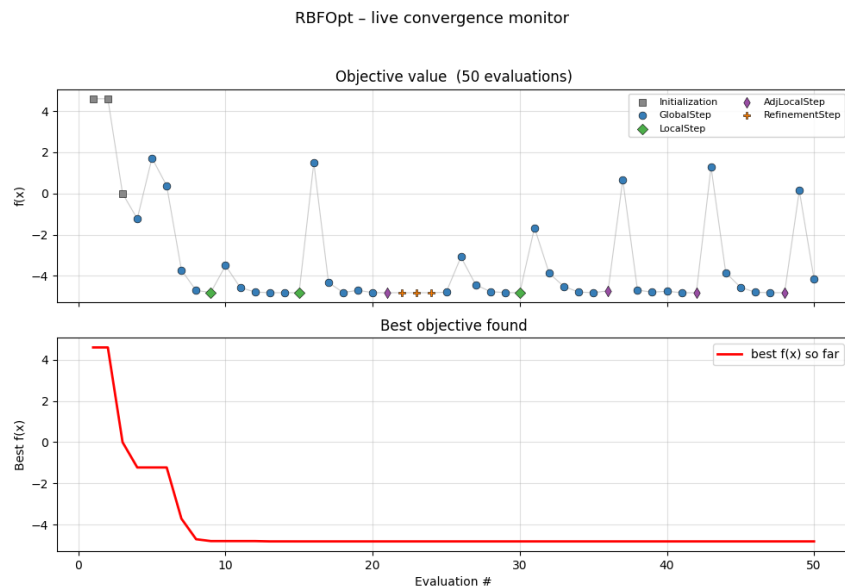
Analytical function vs RBF cubic surrogate



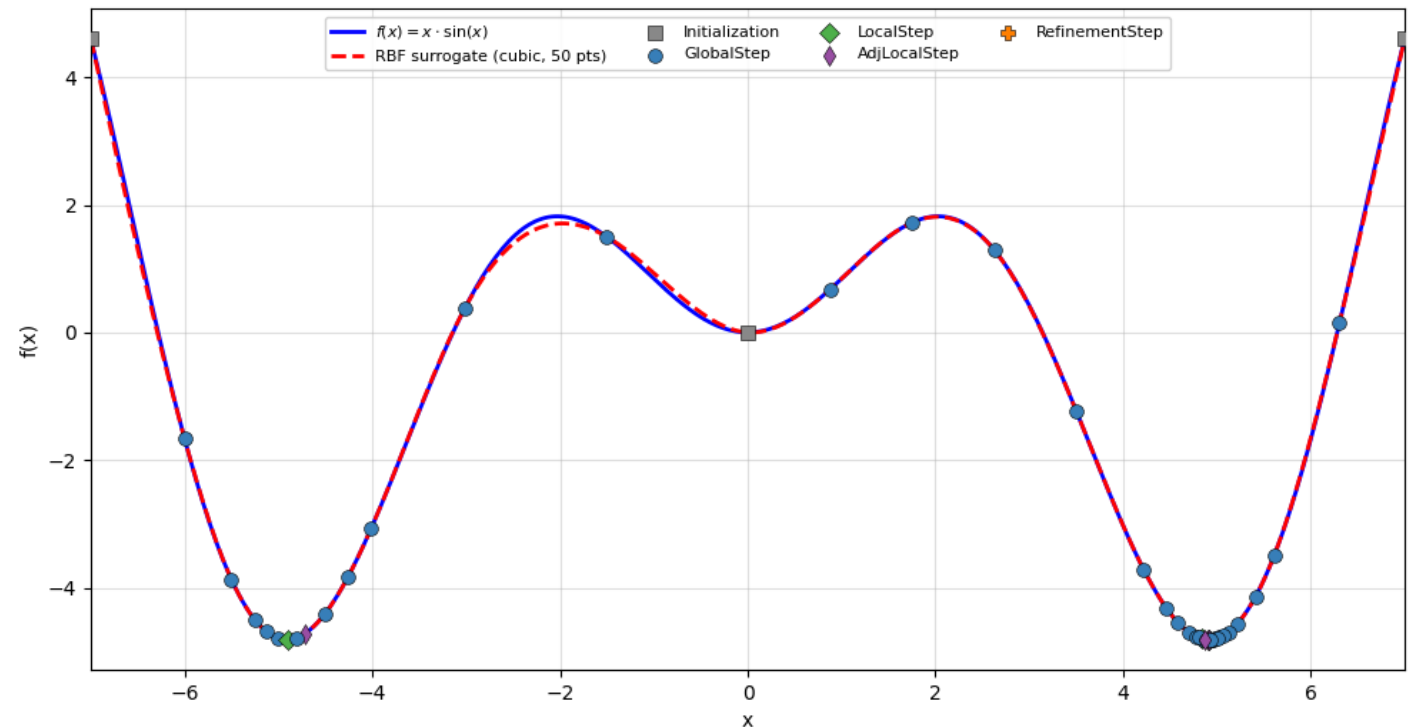
Applicazioni pratiche dell'algoritmo

Esempio 2: minimizzare $y = x \cdot \sin(x)$

NON unicità della soluzione: si sono più minimi assoluti nell'intervallo considerato ma rbfopt ne considererà solo uno!

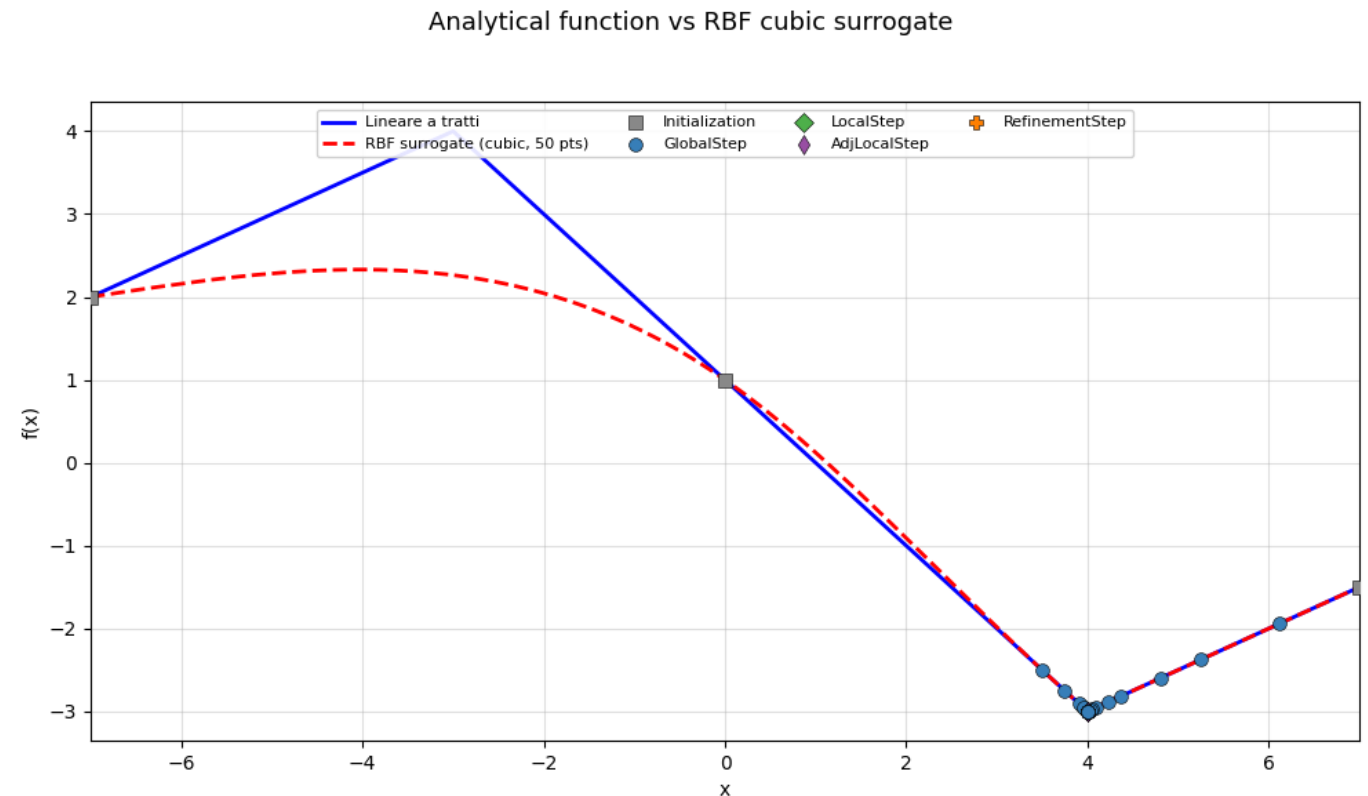
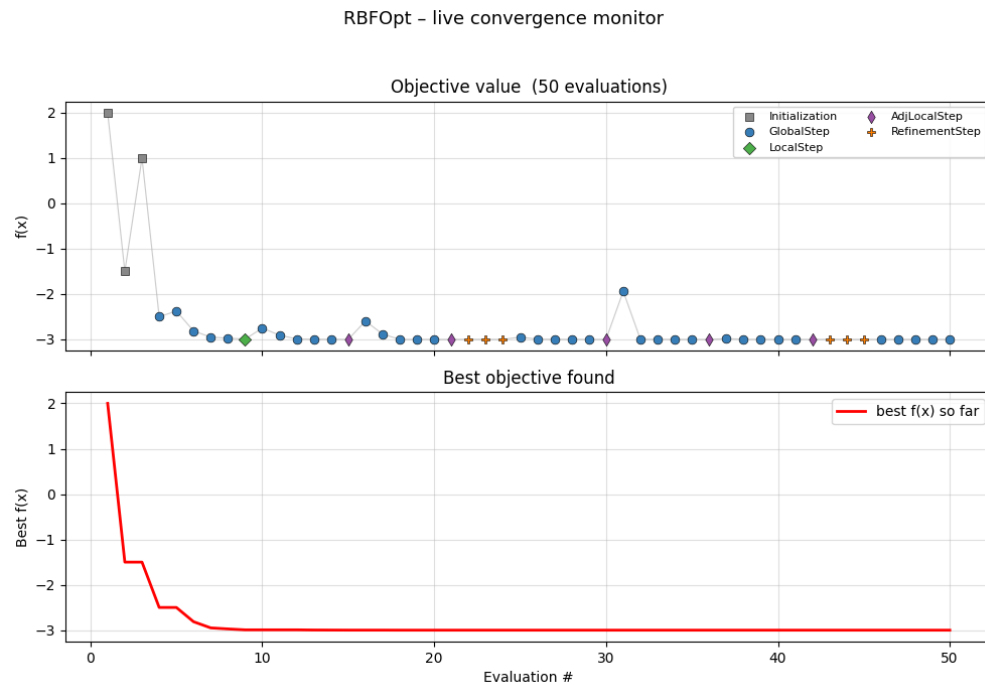


Analytical function vs RBF cubic surrogate



Applicazioni pratiche dell'algoritmo

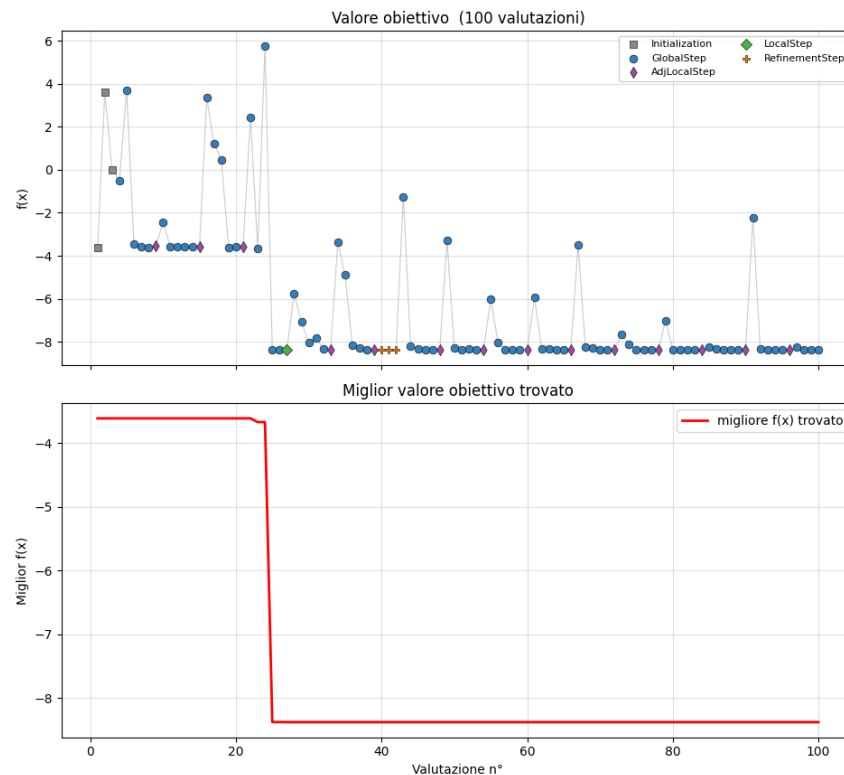
Esempio 3: lineare a tratti, con minimo in un punto di non derivabilità.
Non ricostruisce perfettamente la funzione reale ma l'obiettivo non è questo: è trovare il minimo.



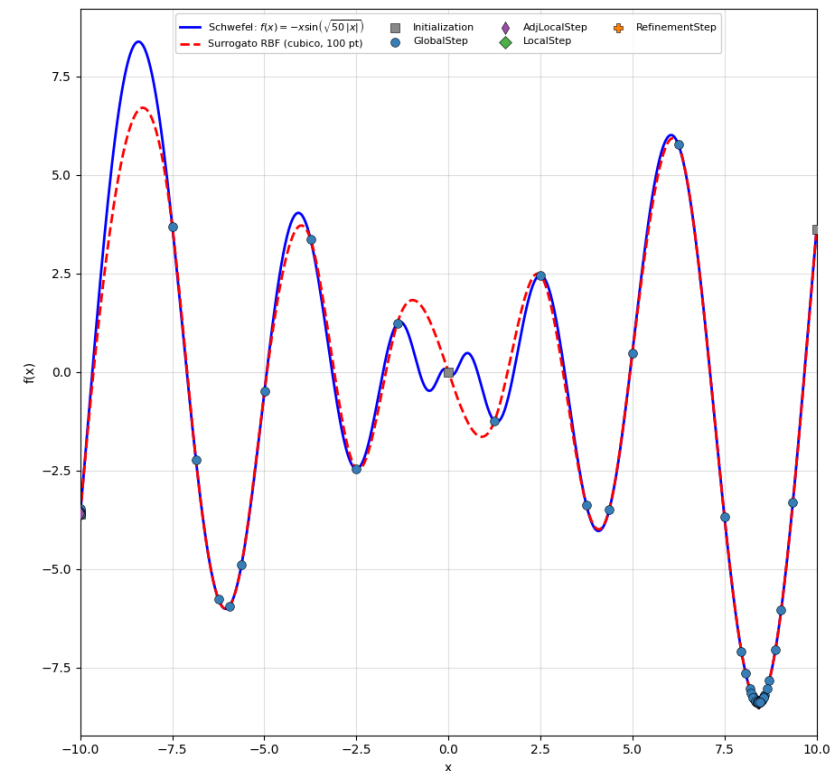
Applicazioni pratiche dell'algoritmo

Esempio 4: funzione di Schwefel 1D, classico benchmark dei problemi di ottimizzazione. Accade infatti che molti metodi si fermano a minimi locali.

RBFOpt - monitor di convergenza in tempo reale



Funzione analitica vs surrogato RBF cubico



Per dubbi o domande, scrivere
a *davide.angelini@polito.it*



**Politecnico
di Torino**

Dipartimento
di Ingegneria Meccanica
e Aerospaziale

